

코드 컨벤션

☼ 상태

완료

TFTgogo_Dev_Guide.pdf

PDF의 Spring Boot / FastAPI / React / 공통 규칙 내용을 기준으로 정리했습니다. 프로젝트 구조는 backend , ai-server , frontend 로 분리된 모노레포 기준입니다.

코드 컨벤션

TFTgogo 프로젝트는 Spring Boot 메인 서버, FastAPI AI 서버, React 프론트엔드를 하나의 모노레포에서 관리한다.

각 영역은 역할을 분리하고, 응답 형식·예외 처리·로깅·API 호출·테스트 규칙을 통일하여 코드 품질과 협업 효율을 높이는 것을 목표로 한다.

1. 컨벤션 요약

영역	핵심 규칙	목적
Spring Boot	예외 처리, 응답 형식, DTO, Swagger, 로깅 규칙 통일	백엔드 코드 일관성 확보
FastAPI	API 계층, 서비스 계층, 모델 계층 분리	AI/RAG 서버 구조 명확화
React	컴포넌트 네이밍, 상태 관리, API 호출 위치 통일	프론트엔드 유지보수성 향상
공통	PR 머지, 환경변수, 테스트, .gitignore 규칙 적용	팀 협업 안정성 확보

2. Spring Boot 코드 컨벤션

항목	규칙
예외 처리	도메인별 Exception 클래스 생성 금지
예외 처리 방식	ErrorCode enum + BusinessException + GlobalExceptionHandler 로 통합 관리

항목	규칙
응답 표준화	모든 API 응답은 <code>ApiResponse</code> 클래스로 통일
반환 타입	<code>ResponseEntity<ApiResponse<T>></code> 사용
단일 조회	<code>null</code> 반환 금지, <code>Optional + orElseThrow()</code> 사용
DTO 규칙	<code>RequestDTO</code> 는 <code>toEntity()</code> 메서드 포함
DTO 규칙	<code>ResponseDTO</code> 는 <code>from()</code> 또는 <code>of()</code> 정적 팩토리 메서드 사용
검증 어노테이션	<code>@NotNull</code> , <code>@NotBlank</code> 는 <code>RequestDTO</code> 에만 사용
로깅	<code>System.out.println</code> 사용 금지
로깅 방식	<code>Log4j2 Logger</code> 사용
로그 레벨	<code>INFO</code> : 정상 흐름, <code>WARN</code> : 비정상 요청, <code>ERROR</code> : 시스템 오류
보안 로그	비밀번호, JWT, 인증번호 등 민감정보 로그 출력 금지

Spring Boot 영역에서는 도메인별 예외 클래스를 따로 만들지 않고, `ErrorCode`, `BusinessException`, `GlobalExceptionHandler` 로 예외 처리를 통합합니다. 또한 모든 API 응답은 `ResponseEntity<ApiResponse<T>>` 형태로 맞추는 것이 기준입니다.

Spring Boot 예외 처리 예시

```
throw new BusinessException(ErrorCode.MEMBER_NOT_FOUND);
```

Spring Boot 단일 조회 예시

```
memberRepository.findById(id)
    .orElseThrow(() -> new BusinessException(ErrorCode.MEMBER_NOT_FOUND));
```

Spring Boot DTO 규칙

DTO 구분	규칙
RequestDTO	<code>toEntity()</code> 메서드 포함
ResponseDTO	<code>from()</code> 또는 <code>of()</code> 정적 팩토리 메서드 사용
RequestDTO	검증 어노테이션 사용 가능
ResponseDTO	검증 어노테이션 사용 금지

```
// RequestDTO
public MemberEntity toEntity() {
    ...
}

// ResponseDTO
public static MemberResponseDto from(MemberEntity entity) {
    ...
}
```

Swagger 규칙

항목	규칙
Swagger 어노테이션	<code>XxxControllerDocs</code> 인터페이스에만 작성
Controller	<code>implements XxxControllerDocs</code> 사용
Controller 메서드	<code>@Override</code> 적용
이름 충돌 방지	Swagger의 <code>ApiResponse</code> 와 커스텀 <code>ApiResponse</code> 이름 충돌 시 풀네임 사용

Swagger 어노테이션은 컨트롤러에 직접 작성하지 않고, `XxxControllerDocs` 인터페이스에만 작성합니다. 컨트롤러는 해당 인터페이스를 구현하고 `@Override` 만 사용하는 방식입니다.

비동기 처리 규칙

항목	규칙
<code>@Async void</code>	사용 금지
반환 타입	<code>CompletableFuture<Void></code> 사용
Executor	<code>SimpleAsyncTaskExecutor</code> 사용 금지
Executor 설정	<code>ThreadPoolTaskExecutor</code> 명시 설정

보안 규칙

항목	규칙
난수 생성	<code>java.util.Random</code> 사용 금지
난수 생성 방식	<code>SecureRandom</code> 사용

항목	규칙
비밀번호 저장	평문 저장 금지
비밀번호 암호화	<code>BCrypt</code> 사용
설정값 주입	<code>@Value</code> 지양
설정 관리	<code>@ConfigurationProperties</code> 사용 권장

3. FastAPI 코드 컨벤션

항목	규칙
<code>api/</code>	라우터, 엔드포인트 정의만 담당
<code>services/</code>	비즈니스 로직, RAG, 추천 알고리즘 처리
<code>models/</code>	Pydantic 요청/응답 스키마 관리
<code>core/</code>	설정, DB 연결, 의존성 주입 관리
응답 표준화	모든 응답은 <code>BaseResponse</code> 스키마 사용
RAG 벡터 검색	<code>pgvector</code> 사용
임베딩 생성	OpenAI Embeddings API 사용
RAG 파이프라인	LangChain 기반 구성
로깅	<code>print()</code> 사용 금지
로깅 방식	Python <code>logging</code> 모듈 사용

FastAPI 영역은 `api`, `services`, `models`, `core` 로 계층을 나누고, 라우터에는 엔드포인트 정의만 두는 구조입니다. RAG 파이프라인은 `pgvector`, OpenAI Embeddings API, LangChain 기반으로 구성합니다.

FastAPI 응답 스키마 예시

```
class BaseResponse(BaseModel):
    success: bool
    message: str
    data: Optional[Any] = None
```

FastAPI 계층 분리 예시

```
# api/router/chat.py
@router.post("/chat")
```

```

async def chat(req: ChatRequest, svc: ChatService = Depends
()):
    return await svc.process(req)

```

```

# services/chat_service.py
class ChatService:
    async def process(self, req: ChatRequest) -> ChatResponse:
        ...

```

4. React 코드 컨벤션

항목	규칙
컴포넌트 파일명	<code>PascalCase</code> 사용
훅 파일명	<code>camelCase</code> 사용
스타일 방식	CSS Modules 사용
Tailwind	사용 금지
전역 상태	Zustand 사용
서버 상태	TanStack Query 사용
로컬 상태	<code>useState</code> 사용
API 호출 위치	<code>api/</code> 폴더에서만 관리
컴포넌트 내부 API 호출	금지
Axios	공통 인스턴스 설정 필수
CSS 클래스명	<code>camelCase</code> 사용
전역 스타일	<code>global.css</code> 에만 작성

React 영역은 컴포넌트 파일명은 `PascalCase`, 훅 파일명은 `camelCase`를 사용합니다. 상태 관리는 전역 상태는 Zustand, 서버 상태는 TanStack Query, 로컬 상태는 `useState`로 구분합니다. API 호출은 컴포넌트 내부에서 직접 하지 않고 `api/` 폴더에서만 관리합니다.

React 컴포넌트 구조 예시

```

components/MemberCard/
├─ MemberCard.jsx

```

└─ MemberCard.module.css

React API 호출 예시

```
// api/matchApi.js
export const getMatch = (name) =>
  axiosInstance.get(`/match/${name}`);
```

CSS Modules 예시

```
/* MemberCard.module.css */
.cardWrapper {
  border-radius: 8px;
}

.memberName {
  font-weight: bold;
}
```

```
import styles from './MemberCard.module.css';

<div className={styles.cardWrapper}>
```

5. 공통 협업 규칙

PR 머지 규칙

브랜치	필요 승인 수	추가 조건
main	3인 이상	팀장 최종 확인 필수
develop	1인 이상	CodeRabbit 리뷰 확인
feature/*	본인 머지 가능	본인 작업 브랜치에 한함

PR 머지 기준은 브랜치별로 다릅니다. `main` 은 3인 이상 승인과 팀장 최종 확인이 필요하고, `develop` 은 1인 이상 승인과 CodeRabbit 리뷰 확인이 필요합니다. `feature/*` 는 본인 작업 브랜치에 한해 본인 머지가 가능합니다.

환경변수 관리

항목	규칙
민감정보	반드시 <code>.env</code> 파일로 관리
GitHub 업로드	<code>.env</code> 업로드 금지
예시 파일	<code>.env.example</code> 은 업로드 가능
<code>.env.example</code> 내용	키 이름만 작성, 실제 값은 비워두기
Spring Boot 설정	<code>application.yml</code> 에서 <code>\${ENV_VAR}</code> 형식으로 참조

테스트 코드 작성 규칙

항목	규칙
테스트 필수 여부	PR 머지 시 담당 Service 테스트 코드 필수
테스트 대상	Service 레이어 단위 테스트
제외 대상	Controller, Entity, DTO, Repository 테스트는 필수 대상 아님
테스트 패턴	<code>given / when / then</code> 패턴 사용
Mock 방식	<code>@ExtendWith(MockitoExtension.class)</code> , <code>@Mock</code> , <code>@InjectMocks</code> 사용
외부 연결	실제 DB, 외부 API 연결 금지
테스트 메서드명	한글 작성 가능

테스트 없는 PR은 머지하지 않는 것을 원칙으로 하고, Service 레이어 테스트를 필수로 작성합니다. 테스트는 실제 DB나 외부 서비스에 연결하지 않고 Mock 기반으로 작성합니다.

테스트 코드 예시

```
@ExtendWith(MockitoExtension.class)
class MemberServiceTest {

    @Mock
    MemberRepository memberRepository;

    @InjectMocks
    MemberService memberService;

    @Test
    void 회원이_없으면_예외를_던진다() {
        // given
```

```

        given(memberRepository.findById(1L)).willReturn(Optional.empty());

        // when & then
        assertThatThrownBy(() -> memberService.getMember(1L))
            .isInstanceOf(BusinessException.class);
    }
}

```

6. .gitignore 필수 항목

구분	제외 대상
공통	.env
공통	*.log , logs/
Spring Boot	backend/src/main/resources/application*.yml
Spring Boot	backend/build/ , backend/.gradle/
FastAPI	ai-server/.venv/ , ai-server/__pycache__/
React	frontend/node_modules/ , frontend/dist/
IDE	.idea/ , .vscode/ , *.iml

7. 코드 작성 체크리스트

체크 항목	확인
API 응답 형식이 통일되어 있는가?	☐
예외 처리가 BusinessException 기준으로 되어 있는가?	☐
null 반환 대신 Optional + orElseThrow() 를 사용했는가?	☐
DTO 변환 규칙을 지켰는가?	☐
System.out.println 또는 print() 를 사용하지 않았는가?	☐
민감정보가 로그에 출력되지 않는가?	☐
Swagger 어노테이션을 Docs 인터페이스에만 작성했는가?	☐
React API 호출이 api/ 폴더에서만 이루어지는가?	☐
CSS Modules 클래스명이 camelCase 인가?	☐

체크 항목	확인
Service 레이어 테스트 코드를 작성했는가?	☐
<code>.env</code> 파일이 GitHub에 올라가지 않는가?	☐
CodeRabbit 리뷰를 확인했는가?	☐